Learning Objectives

Course: CRE-201 — AI for CRE

Duration: 4-day, two weekends (Sat-Sun + week gap + Sat-Sun; ~17 hours total, ~12 hours core instruction + ~5 hours extended practice & bookends) **Target Audience:** CRE professionals (brokers, property managers, asset managers, analysts) **Prerequisite:** Basic familiarity with single-turn desktop chatbots (e.g., ChatGPT, Claude). You should have used a chatbot enough that basic prompting is comfortable — typically 10+ sessions for real tasks, not just casual experimentation. No coding or technical background required.

Overview

By the end of this workshop, participants will be able to apply agentic engineering principles — through a management lens — to design, build, and refine multi-step AI workflows for commercial real estate tasks using Claude Code.

Pedagogical Approach: Every agentic engineering concept is taught through a direct CRE parallel. The documentation chain is distributed across the objectives where each document naturally fits (see §Documentation Chain Distribution below). Participants are not learning foreign skills — they are recognizing familiar ones applied to AI, while building genuinely new competencies where the CRE parallel reaches its limits. This mapping is established in Objective 1 and reinforced throughout. See `04-materials/CRE_Metaphor_Map.md` for the complete mapping reference.

Frameworks taught: Six core frameworks structure the course:

1. **Holy Trinity** — The three roles of agentic engineering (PM, Designer, Engineer)
2. **Four Pillars** — The four factors that determine AI output quality (Context, Model, Prompt, Tools)
3. **8-Step Process** — The engineering lifecycle from discovery to documentation
4. **Documentation Chain** — Four planning documents (PRD, ADR, DDD, TDD) distributed across the course

5. **Claude Code Basics** — Four component types (Agents, Skills, Slash Commands, Plugins) with the capstone deliverable being a self-contained CRE workflow plugin
6. **Value-Delivery Fit** — Paired VPD (business + engineering fit) with BMC awareness (cost sniff test), grounding the documentation chain before OFD/PRD

Industry vocabulary note: The six frameworks above collectively implement what the industry calls the three disciplines of AI work — *Prompt Engineering*, *Context Engineering*, and *Intent Engineering*. The course progresses from era 1 through era 3. See Level 0 §Three-era framing for the definitions and course-arc mapping.

Anchor Example — Lease Abstraction Plugin: A single worked example threads through every objective: a classification-gated lease abstraction plugin that extracts key terms from commercial leases. The instructor builds this plugin progressively across the course using a "Gradual Release of Responsibility" model (I do → We do → You do):

- **"I do" (Level 0 – Obj 6):** The instructor leads each concept applied to lease abstraction. Participants replicate on their own machines — Claude Code is installed before Day 1 and open from the first session. Even during instructor-led demonstrations, participants follow along side-by-side: building VPD diagrams in Claude Code, overloading a context window to see token limits firsthand, watching how the same prompt produces different outputs on different machines. The side-by-side pattern means participants see LLM variance from Day 1 — reinforcing that specification, not luck, determines output quality.
- **"We do" (Obj 7 – Obj 9):** Participants build alongside the instructor — starting with the productive failure exercise (everyone types the same poorly-specified prompt, compares their different-but-similarly-flawed outputs), then applying concepts to the lease abstraction plugin (annotating ADR/DDD, diagnosing failures, writing acceptance criteria).
- **"You do" (Obj 10):** Participants build their own CRE workflow plugin independently, having internalized the full pipeline from the worked example.

This design leverages the Worked Example Effect (Sweller): studying a worked example before attempting equivalent problem-solving reduces cognitive load and produces better transfer. By the capstone, students have seen every concept applied end-to-end — the capstone asks them to replicate the *pattern*, not discover it from scratch.

Why lease abstraction? Every CRE professional understands the task. It naturally exposes every agentic engineering challenge taught in the course: classification (office vs. industrial vs. residential), dynamic context loading (jurisdiction-specific legislation,

property-type templates), context overflow management (50-page leases + statutes exceed practical context windows), multi-step decomposition, quality judgment (did it extract the right terms?), and frontier recognition (when does a clause require human interpretation?). Each objective's `**Lease Abstraction Thread**` section shows how the instructor applies that objective's concepts to this running example.

Note on management skills: Context assembly, task decomposition, quality judgment, iterative refinement, workflow integration, and frontier recognition are taught as the applied skills within the objectives — not as a separate framework. They emerge naturally from the objectives that develop them. See the Management Skills Integration Map below for where each skill appears.

Level 0: Orientation (20 min, Day 1 opening)

Purpose: Establish baseline AI vocabulary and demonstrate the gap between single-turn prompting and multi-step workflows. Reduces the cognitive jump required when Objectives 1-2 introduce new frameworks.

Content: - Live demonstration: single-turn prompt vs. structured multi-step workflow for the same CRE task — lease abstraction (5 min). The instructor pastes a lease excerpt into a chatbot with "summarize the key terms" (single-turn), then shows the same task handled by a multi-step workflow that classifies the property type, loads relevant legislation, extracts sections systematically, and validates against a template. The quality gap is immediately visible. This introduces the lease abstraction anchor example that threads through every objective. - Key vocabulary: prompt, context, model, agent, workflow, iteration (5 min) - Three-era framing (2 min): The instructor names the three evolving disciplines of AI work — *Prompt Engineering* (crafting individual instructions, where most people are stuck), *Context Engineering* (managing what the AI knows, now an industry standard), and *Intent Engineering* (encoding your goals, constraints, and decision boundaries so the AI knows how to act, not just what to know). "This course teaches all three. By Day 4, you'll operate across the full stack — from crafting prompts, to assembling context, to defining intent that makes your workflows converge on the right answer." This framing maps directly to the course arc: Obj 2 (Four Pillars, including Prompt and Context), Obj 6 (Context Assembly), and Obj 4 (OFD as intent). - "Where are you now?" self-assessment: participants rate their current AI usage on a 4-point scale (8 min)

Assessment: None (orientation only) **Affective goal:** Reduce anxiety by showing that the gap between current usage and structured workflow is bridgeable, not a cliff.

Learning Objectives

> ***Teaching Notes:*** *Detailed teaching sequences, lease abstraction threads, CRE parallels, and worked examples for each objective are in teaching-notes.md.*

Objective 1: The Holy Trinity of Agentic Engineering

Objective Statement: Explain the three roles of agentic engineering (Product Manager, Engineer, Designer) and how each maps to management competencies CRE professionals already possess.

Cognitive Level: Understand **Time Allocation:** 40 min **Assessment Method:** Written explanation connecting each role to a specific CRE management parallel **Assessment Type:** Formative (peer discussion, no instructor grading) **Module:** Weekend 1, Day 1

Rationale: Establishes the foundational mental model that agentic engineering is management work — not coding — by anchoring new concepts to existing professional identity. CRE professionals already act as PMs (scoping deals), engineers (building analyses), and designers (shaping client deliverables).

Objective 2: The Four Pillars

Objective Statement: Describe the four pillars of agentic engineering (Context, Model, Prompt, Tools) and explain how each pillar contributes to the quality of AI-generated output.

Cognitive Level: Understand **Time Allocation:** 40 min **Assessment Method:** Diagram or concept map showing pillar interdependencies with CRE-specific examples for each **Assessment Type:** Formative (self-assessed against instructor example) **Module:** Weekend 1, Day 1

Rationale: Provides the diagnostic framework participants will use throughout the course. When AI output is wrong, one or more pillars are misconfigured. Understanding the pillars before applying them prevents trial-and-error frustration.

These Four Pillars return in Objective 3 as the engineering capability substrate — the same components viewed from a design lens rather than a diagnostic lens.

— Integration Pause 1 (15 min) —

Participants map all three Holy Trinity roles and all four Pillars to a single CRE task from their own work. Purpose: consolidate before the afternoon adds more frameworks. This exercise surfaces which participants have strong mental models and which need additional scaffolding.

Objective 3: Value-Delivery Fit (Paired VPD Lite + BMC Awareness)

Objective Statement: Apply a paired value-delivery fit analysis to determine whether a proposed CRE workflow is worth building: identify the user jobs and pains the workflow addresses (business fit), recognize that the Four Pillars from Objective 2 are the engineering capability substrate that must deliver against those jobs (engineering fit), validate cross-layer traceability between business claims and engineering mechanisms, and recognize that full business viability (cost structure, runtime economics) requires a broader canvas analysis introduced here and developed fully in the standalone APD series.

Cognitive Level: Apply **Time Allocation:** 75 min (60 min paired VPD + 15 min BMC awareness) **Assessment Method:** Paired VPD worksheet (business customer profile + value map; engineering system task profile + delivery map) with cross-layer trace annotations. Plus 5-min cost sniff test (build cost, per-use value, break-even). **Assessment Type:** Formative (peer review using cross-layer traceability checklist) **Module:** Weekend 1, Day 1 (after Integration Pause 1)

Rationale: The Four Pillars (Obj 2) teach "what determines AI output quality" as a diagnostic tool. This objective reframes the same four components — Context, Model, Prompt, Tools — as the engineering capability substrate: the resources the system needs to deliver value. This bridge eliminates the perception of "another framework" and reveals the larger system the Four Pillars were always part of.

Before participants learn *how* to build (8-Step Process, Obj 4), they need to validate *whether* to build. The paired VPD asks two questions: (1) Does this workflow address real user jobs and pains? (Business VPD) (2) Can our system actually deliver the promised value? (Engineering VPD). Cross-layer traceability — "if a business pain reliever has no engineering mechanism, it's marketing copy" — is practiced on a worksheet.

The BMC awareness segment (15 min) introduces the viability dimension without teaching the full canvas. Participants do a 5-min cost sniff test (build cost, per-use value, break-even) and learn the full cost principle: token cost is never the whole cost — include human review time, failure/rework probability, and cost of delay. The full four-canvas system (paired VPDs + paired BMCs) is the standalone APD series.

Objective 4: The 8-Step Process + OFD/PRD

Objective Statement: Apply the 8-step agentic engineering process (Discover, Plan, Design, Structure, Implement, Validate & QA, Test, Document) to plan an agentic workflow for a specific CRE task, first defining an objective function (what success looks like, what's off limits, when to stop) and then producing a PRD that operationalizes those objectives into buildable requirements.

Cognitive Level: Apply **Time Allocation:** 90 min **Assessment Method:** Completed 8-step planning worksheet for a participant-selected CRE task, with an objective function (primary objective, hard constraints, escalation triggers) and a PRD that serves the objective function (problem statement, users, requirements, constraints, success metrics) **Assessment Type:** Formative (peer review using OFD/PRD checklist) **Module:** Weekend 1, Day 2

Rationale: The 8-step process is the backbone methodology. The objective function and PRD are introduced here — not in a standalone documentation module — because they are the natural output of the Discover and Plan steps. Participants produce the first bookend of the documentation chain while learning the overall process.

Intent engineering connection: The OFD is, in industry terms, *intent engineering* — encoding organizational purpose into machine-actionable parameters. The three OFD fields (primary objective, hard constraints, escalation triggers) map to intent parameters: trade-off logic, decision boundaries, and resolution hierarchies respectively. See `teaching-notes.md` §Objective 4 for the full teaching sequence, OFD-as-intent table, and the Microsoft Copilot motivating example.

Objective 5: Task Decomposition

Objective Statement: Apply task decomposition to break a complex CRE process into discrete, delegable units suitable for agentic execution, identifying human checkpoints at

appropriate boundaries.

Cognitive Level: Apply **Time Allocation:** 50 min **Assessment Method:** Decomposition map showing a multi-step CRE process broken into AI-delegable and human-required components **Assessment Type:** Formative (peer review) **Module:** Weekend 1, Day 2

Rationale: Task decomposition is the single most important applied skill for agentic engineering. CRE processes (lease renewals, asset reports, due diligence) are inherently multi-step. Knowing where to split them determines whether AI helps or creates chaos.

— Integration Pause 2 (15 min) —

Participants connect the Holy Trinity, Four Pillars, 8-Step Process, and Task Decomposition to the same CRE task they've been developing throughout the day. By this point, they should be able to articulate: "I'm acting as [role], following [step], and this task decomposes into [units]." Purpose: consolidate all frameworks covered so far into a single integrated mental model before adding Context Assembly.

Objective 6: Context Assembly

Objective Statement: Apply context assembly techniques to provide Claude Code with sufficient background information, constraints, and success criteria for accurate CRE task execution.

Cognitive Level: Apply **Time Allocation:** 50 min **Assessment Method:** Side-by-side comparison of AI outputs from a thin-context prompt vs. a fully assembled context prompt for the same CRE task **Assessment Type:** Formative (self-assessed — the output quality difference is self-evident) **Module:** Weekend 1, Day 2 (after Integration Pause 2)

Rationale: Context is the pillar CRE professionals most underestimate. They know their domain deeply but fail to transfer that knowledge to the AI. This objective directly addresses the gap between "I know what I want" and "the AI produced garbage."

Objective 7: Claude Code Fundamentals + ADR/DDD

Objective Statement: Use core Claude Code components (agents, skills, slash commands, plugins) to execute a guided multi-step CRE workflow, distinguishing the purpose and appropriate use case for each component type. Interpret instructor-provided ADR and DDD examples to understand how design decisions and domain models structure a real workflow. Understand that a plugin is the self-contained distributable unit that bundles agents, skills, and commands into a single package — the format participants will use for their capstone deliverable.

Cognitive Level: Apply **Time Allocation:** 90 min **Assessment Method:** Completion of a structured lab exercise where participants use at least three of the four Claude Code component types (agents, skills, slash commands, plugins) to process a CRE task, and examine a minimal plugin structure (`plugin.json` manifest + at least one skill). Participants also annotate an ADR and DDD with "I would have made this decision because..." or "I would have modeled this differently because..." to demonstrate comprehension. **Assessment Type:** Formative (lab completion + peer discussion of ADR/DDD annotations) **Module:** Weekend 2, Day 3

Rationale: Bridges from conceptual understanding to hands-on capability. The ADR and DDD are taught here — not in a standalone documentation module — because they make the most sense in the context of a real Claude Code workflow. The ADR explains *why* the lab exercise was designed this way (which components, which approach). The DDD shows *how* the domain was modeled for the exercise. Participants see these documents as living artifacts that structure real work, not abstract templates.

Objective 8: Diagnosing Failures & Frontier Recognition

Objective Statement: Analyze a failed or subpar AI output to diagnose which of the four pillars (Context, Model, Prompt, Tools) requires adjustment, and differentiate between CRE tasks suitable for agentic automation and those requiring human judgment.

Cognitive Level: Analyze **Time Allocation:** 75 min **Assessment Method:** (Part A) Given three pre-generated AI outputs with different failure modes, correctly identify the failing pillar and propose a fix for each. (Part B) Classification exercise sorting 12-15 CRE tasks into categories (fully automatable, AI-assisted with human review, human-only) with written justification for boundary cases. **Assessment Type:** Part A: Formative (peer-

scored). Part B: Formative (instructor-facilitated group discussion — boundary cases are deliberately arguable). **Module:** Weekend 2, Day 3

Rationale: These two analytical skills are complementary faces of the same judgment: understanding where and why AI fails. Pillar diagnosis answers "what went wrong?" Frontier recognition answers "should I even have tried?" Combining them into a single module creates a more coherent analytical block: diagnose failures in things AI *should* be able to do (pillar diagnosis), then identify things AI *shouldn't* attempt (frontier recognition). The boundary between the two is itself instructive.

— Integration Pause 3 (15 min) —

Participants revisit their Weekend 1 task decomposition and identify: (1) which decomposed units could fail and why (pillar diagnosis preview), and (2) which units are beyond the AI frontier and need human execution. Purpose: connect the analytical skills just learned to their own workflow design.

Objective 9: Iterative Refinement, Quality Judgment & TDD

Objective Statement: Evaluate AI-generated CRE work products using quality judgment criteria derived from TDD-style acceptance criteria defined before execution, applying iterative refinement across multiple cycles to improve output until it meets professional standards.

Cognitive Level: Evaluate **Time Allocation:** 60 min **Assessment Method:** Multi-round refinement exercise where participants: (1) write 3-5 acceptance criteria for a CRE deliverable before the AI produces it, (2) evaluate the initial AI output against those criteria, (3) provide specific feedback and iterate through 2-3 cycles, documenting quality improvement at each round. **Assessment Type:** Summative (instructor reviews acceptance criteria quality and refinement log for capstone readiness) **Module:** Weekend 2, Day 4

Rationale: TDD acceptance criteria are the mechanism for quality judgment — they define "what good looks like" before execution, giving participants an objective standard to evaluate against. This objective teaches TDD not as an abstract document but as the practical tool that makes iterative refinement productive rather than aimless. The review-refine-iterate loop is where agentic engineering produces professional-grade results.

Objective 10: Capstone — CRE Workflow Plugin

Objective Statement: Build a self-contained Claude Code plugin for a recurring CRE workflow task, integrating the OFD/PRD design (from Obj 4), at least one skill, TDD acceptance criteria (from Obj 9), and a frontier boundary map identifying where AI stops and human judgment starts. Present the plugin design and working demonstration to peers for feedback.

Cognitive Level: Apply/Create (template-guided synthesis) **Time Allocation:** 90 min **Assessment Method:** Participants present a working plugin package that includes: (1) a `plugin.json` manifest defining the plugin's name, description, and components, (2) at least one skill implementing a CRE workflow step, (3) an OFD/PRD scoping the plugin (refined from Day 1), (4) TDD acceptance criteria (refined from Obj 9), (5) a frontier boundary map, and (6) a live demonstration using `--plugin-dir` showing the plugin in action. Peer feedback using a structured rubric. **Assessment Type:** Summative (instructor evaluates design coherence and plugin functionality; peer feedback using structured rubric) **Module:** Weekend 2, Day 4 — Capstone

Rationale: Synthesis of all course content into a single, tangible, *executable* deliverable participants take back to work. The capstone is a **self-contained Claude Code plugin** — a directory with a manifest and at least one skill that solves a real CRE task. This is a significant upgrade from a design document: participants leave with something that *runs*, not just something that describes. The capstone plugin design starts with a VPD fit check (from Obj 3): does this workflow address validated user jobs and pains, and can the engineering substrate deliver? The plugin is explicitly framed as a **working first draft** — functional but requiring 2-4 additional hours of refinement (adding more skills, tuning prompts, expanding the OFD) before it is production-ready. This honest framing sets realistic expectations while producing genuine value: participants leave with a working plugin they can refine independently, not a polished artifact they'll never touch again.

Affective Domain Objectives

In addition to the cognitive objectives above, the course targets two affective outcomes (Krathwohl's taxonomy):

Affective Objective A: Willingness to Experiment

By the end of Weekend 1, participants will demonstrate willingness to experiment with Claude Code for CRE tasks without fear of "breaking" something or producing embarrassing output.

Indicators: Participants attempt exercises without excessive hesitation. Participants share failed outputs with peers without defensiveness. Participants ask "what if I tried..." questions during lab time.

Design mechanisms: The Level 0 orientation normalizes imperfect output. The competence arbitrage framing builds psychological safety. Integration pauses create low-stakes practice moments.

Affective Objective B: Confidence in Diagnostic Ability

By the end of the course, participants will express confidence in their ability to diagnose and fix AI output failures independently, and to distinguish between tasks they should and should not delegate to AI.

Indicators: Participants use pillar language ("this is a context problem") rather than generic frustration ("it doesn't work"). Participants articulate specific corrective actions rather than abandoning the tool. Participants make frontier judgments ("this needs human review") with supporting reasoning.

Design mechanisms: The pillar diagnostic framework gives participants a vocabulary for failure that replaces helplessness with agency. The frontier recognition exercise builds calibrated confidence.

Objective Summary Table

#	Objective	Cognitive Level	Time	Assessment	Type
0	Level 0 Orientation	—	20 min	Self-assessment	—
1	Holy Trinity roles → CRE management parallels	Understand	40 min	Written explanation	Formative
2	Four Pillars and their interdependencies	Understand	40 min	Concept map	Formative
—	Integration Pause 1	—	15 min	Consolidation exercise	—
3	Value-Delivery Fit (Paired VPD Lite + BMC Awareness)	Apply	75 min	Paired VPD worksheet + cost sniff test	Formative
4	8-Step Process + OFD/PRD for a CRE task	Apply	90 min	Planning worksheet + OFD/PRD	Formative
5	Task Decomposition with human checkpoints	Apply	50 min	Decomposition map	Formative
—	Integration Pause 2	—	15 min	Consolidation exercise	—
6	Context Assembly for accurate AI execution	Apply	50 min	Before/after output comparison	Formative
7	Claude Code components + ADR/DDD interpretation	Apply	90 min	Structured lab + annotation	Formative
8	Diagnosing Pillar Failures + Frontier Recognition	Analyze	75 min	Failure diagnosis + task classification	Formative
—	Integration Pause 3	—	15 min	Consolidation exercise	—

#	Objective	Cognitive Level	Time	Assessment	Type
9	Iterative Refinement + Quality Judgment + TDD	Evaluate	60 min	Acceptance criteria + refinement log	Summative
10	Capstone: CRE Workflow Plugin	Apply/Create	90 min	Working plugin + presentation	Summative

Core instruction (Obj 1-10): 660 min (11.0 hours) **Integration pauses:** 45 min **Level 0 orientation:** 20 min **Total core structured time:** 725 min (12.1 hours) **Extended practice & bookends (two-weekend format):** ~265 min (4.4 hours) **Total structured time (all 4 days):** ~990 min (16.5 hours) **Note:** Core time allocations represent minimum instructional depth per objective. The two-weekend format adds extended practice sessions (VPD Refinement Workshop, Build-Ready Checkpoint, Extended Lab, Capstone Polish), bookends (welcome, recaps, wrap-ups, closing), and a between-weekend homework assignment. See `02-design/course-outline.md` for the full 4-day schedule.

Cognitive Level Distribution

- **Understand:** 2 objectives (20%)
- **Apply:** 5 objectives (50%)
- **Analyze:** 1 objective (10%)
- **Evaluate:** 1 objective (10%)
- **Apply/Create:** 1 objective (10%)

Total: 10 objectives + Level 0 orientation + 3 integration pauses

Note on Bloom's levels: The original design had 11 objectives across five Bloom's levels. The revised design achieves broader coverage with 10 objectives by combining two Analyze-level objectives into one stronger block (Obj 8) and embedding documentation chain artifacts into the objectives where they naturally belong. Objective 3 (Value-Delivery Fit) was added to ground the documentation chain in validated business and engineering fit. The capstone is reclassified from pure Create to Apply/Create, reflecting the realistic outcome of template-guided synthesis under time pressure.

Scaffolding Plan

Learning Progression:

The objectives build in a deliberate sequence with explicit consolidation points.

Weekend 1, Day 1 (Saturday): Foundations & Mental Models (~4 hours structured)

- **Level 0 (20 min):** AI vocabulary baseline, live demo of single-turn vs. multi-step
- **Morning: Mental models** — Holy Trinity (Obj 1), Four Pillars (Obj 2)
- **Integration Pause 1 (15 min):** Map both frameworks to a single CRE task
- **Late morning: Value-delivery fit** — Paired VPD + BMC Awareness (Obj 3)
- **Extended practice:** VPD Refinement Workshop (30 min)

Weekend 1, Day 2 (Sunday): Process & Design (~4.5 hours structured)

- **Morning: Core process** — 8-Step Process + OFD/PRD (Obj 4)
- **Late morning: Decomposition & context** — Task Decomposition (Obj 5)
- **Integration Pause 2 (15 min):** Connect all frameworks to the same CRE task
- **Early afternoon:** Context Assembly (Obj 6)
- **Extended practice:** Build-Ready Checkpoint (35 min) — verify VPD → OFD/PRD → decomposition → context strategy chain
- **Homework briefing:** Try building a simple skill for a CRE task during the week (Claude Code already installed from Day 1)

Week gap (5 business days): Participants apply Weekend 1 frameworks to real work and attempt Claude Code experimentation. The productive struggle during the week makes Weekend 2's Claude Code lab and failure diagnosis resonate.

Weekend 2, Day 3 (Saturday): Tools & Analysis (~4.25 hours structured)

- **Opening: Week-in-Review (20 min):** Participants share what they tried, what worked, what failed
- **Morning: Hands-on tools** — Claude Code Fundamentals + ADR/DDD (Obj 7), including productive failure demo
- **Late morning: Analytical skills** — Diagnosing Failures & Frontier Recognition (Obj 8)
- **Integration Pause 3 (15 min):** Revisit Day 2 decomposition with new analytical lens

- **Extended lab:** Failure Diagnosis on Own Workflows (45 min) — authentic assessment using participants' real week-gap experience

Weekend 2, Day 4 (Sunday): Quality & Synthesis (~4.5 hours structured)

- **Morning: Quality and refinement** — Iterative Refinement + TDD (Obj 9)
- **Late morning: Capstone build** — CRE Workflow Plugin (Obj 10, 90 min)
- **Early afternoon: Capstone polish** (45 min) — iterative refinement against own TDD criteria
- **Capstone presentations** (45 min) — live demos and peer rubric feedback
- **Course closing** (20 min) — reverse transfer exercise (5 min: "Name one human-team management practice you'll change based on what you learned here"), next steps, post-course reinforcement plan

Weekend 2 CRE mapping note: By Weekend 2, reduce the introduction of new CRE parallels. Participants should be transitioning to agentic engineering vocabulary natively. Reinforce established mappings rather than introducing new ones. The Claude Code component mappings (agents = subcontractors, etc.) are the last new CRE parallels introduced.

Management Skills Integration Map

Each skill emerges naturally from the objective that develops it:

Management Skill	Primary Objective	Supporting Objectives	Where It Appears
Value-Delivery Fit	3 (primary)	4 (OFD grounded in VPD), 9 (economics check)	Day 1
Context Assembly	6 (primary)	4 (PRD scope), 7 (DDD domain model)	Day 2, Day 3
Quality Judgment	9 (primary)	8 (pillar diagnosis), 10 (capstone review)	Day 3, Day 4
Task Decomposition	5 (primary)	4 (8-step planning), 10 (capstone)	Day 2, Capstone
Iterative Refinement	9 (primary)	6 (context iteration)	Day 2, Day 4
Workflow Integration	10 (primary)	4 (8-step end-to-end)	Day 2, Capstone
Frontier Recognition	8 (primary)	5 (human checkpoint identification)	Day 3

Documentation Chain Distribution

The documentation chain forms a sequential set of artifacts where each reduces entropy for the next. **The OFD anchors the entire chain** — it defines what success looks like before any other document is written. At the 201 level, the OFD is physically embedded as the first section of the PRD but taught first in sequence; at 301, it separates into a standalone document and the chain expands. Each document in the chain translates abstract organizational purpose into increasingly concrete, machine-actionable parameters — from "is this worth doing?" (VPD) through "what does good look like?" (OFD) to "did we achieve it?" (TDD). Rather than teaching the chain in a single compressed block, documents are distributed across the objectives where each naturally fits:

```
201 Chain (VPD-grounded, OFD-first):
VPD → OFD/PRD → ADR → DDD → TDD → Implementation
  ↓   ↓   ↓       ↓       ↓       ↓
  Fit? ||  What   Which    How    Done?
  for  |  & Why  & Why   (shared  (pass/
  whom?|         this    lang)   fail)
       |         way
       ↓
      Optimizing    ← The OFD grounds the chain.
      for what?       TDD at the end evaluates
      Off limits?     against OFD at the beginning.
      When to stop?   VPD validates fit before OFD.

301 Chain (OFD separates, TDD splits):
OFD → PRD → ADR → DDD → TDD + Evals Spec → Implementation
 ↓      ↓      ↓      ↓       ↓        ↓
Score- What   Which   How   Correct?  Good?
board  & Why  & Why  (shared (binary   (threshold
+             this   lang)  pass/     quality
guard         way           fail)     bars)
rails
```

Document	Objective	How Taught	Participant Activity
VPD (Value-Delivery Fit)	Obj 3 (paired exercise)	Paired VPD worksheet, cross-layer traceability	Complete Business + Engineering VPD with trace annotations
OFD (Objective Function Document)	Obj 4 (taught first)	OFD fields completed before PRD template	Define primary objective, hard constraints, escalation triggers
PRD (Product Requirements Document)	Obj 4 (serves the OFD)	Produced as the Discover/Plan output	Write a PRD that operationalizes the OFD into buildable requirements
ADR (Architectural Decision Record)	Obj 7 (Claude Code Lab)	Instructor-provided worked example	Interpret and annotate
DDD (Domain Data Dictionary)	Obj 7 (Claude Code Lab)	Instructor-provided worked example	Interpret and annotate
TDD (Test-Driven Development)	Obj 9 (Quality Judgment)	Produced as acceptance criteria	Write both binary (TDD) and rubric-based (EDD) acceptance criteria
EDD (Evaluation-Driven Development)	Obj 9 (awareness only)	Distinction named; both criteria types written together	Recognize which of their criteria are deterministic vs. judgment-based

Chain literacy sequence: Participants define the objective function first (OFD fields in Obj 4), then write the PRD to serve it, and finally write acceptance criteria (TDD in Obj 9) that trace back to the OFD. The two bookend documents — OFD at the beginning, TDD at the end — are independent anchors: the OFD defines "what are we optimizing for?" and the TDD evaluates "did we achieve it?" This independence is what prevents self-referential validation — the evaluation criteria (TDD) are anchored to the objective function (OFD), not to whatever the implementation produced.

Participants also interpret the two middle documents (ADR, DDD in Obj 7). The OFD's escalation triggers connect forward to Frontier Recognition (Obj 8) — participants define the escalation boundary in writing before they encounter it in practice. At 201, TDD and EDD criteria are written together as "acceptance criteria" with the distinction named but not operationalized. Full production of all documents as standalone artifacts — including a separated OFD with full metric hierarchy, a deterministic TDD test harness, and a

separate Evals Spec (the EDD artifact) with datasets, rubrics, and graders — is a 300-level skill.

Chain ↔ 8-Step Process Mapping

Documentation Chain	8-Step Process Stages	What Gets Resolved
OFD (taught first)	Discover	Optimization target defined, guardrails set, escalation boundary drawn
PRD (serves the OFD)	Discover + Plan	Problem scoped, requirements operationalize OFD into buildable spec
ADR	Design	Key technical/architectural decisions made and recorded
DDD	Structure	Domain modeled, shared language established
TDD	Validate & QA + Test	Deterministic acceptance criteria defined, binary pass/fail determined
EDD (Evals Spec at 301)	Validate & QA + Test	Probabilistic quality thresholds defined, rubric-based evaluation
(Implementation)	Implement + Document	Build the workflow, record what was built

Chain ↔ Four Pillars Mapping

Document	Context	Model	Prompt	Tools
VPD	Capability substrate	Model capability needs	Behavioral controls needed	Action surface needed
OFD (taught first)	Success criteria, hard constraints	Model capability vs. objective	Escalation triggers	Budget limits (tokens, tool calls)
PRD (serves the OFD)	Domain background, constraints	-	Requirements (what to build)	-
ADR	-	Which AI model for which task	-	Which integrations/platforms
DDD	Domain language, entity relationships	-	Implementation language (verbatim)	-
TDD	Success criteria context	-	Binary acceptance criteria (what "correct" means)	Deterministic test tooling
EDD (Evals Spec at 301)	Quality standards, task distributions	Model capability assessment	Quality thresholds (what "good" means)	Eval harness, graders, datasets

Assessment Philosophy

Formative assessments (Objectives 1-8): Designed for feedback and learning, not grading. Peer review, self-assessment, and instructor-facilitated discussion. The instructor monitors comprehension through observation and spot-checks, not through systematic evaluation of 20 individual submissions. This is realistic for a 4-day workshop with 12-20 participants.

Summative assessments (Objectives 9-10): The only assessments that receive structured instructor evaluation. Objective 9 (acceptance criteria + refinement log) serves as a readiness check for the capstone. Objective 10 (capstone presentation) is evaluated against a structured rubric focused on design coherence, not polish.

Peer assessment: Used throughout for feedback richness. Peers provide perspectives the instructor can't (different CRE roles, different task contexts). Peer feedback uses structured prompts ("Does this decomposition identify the right human checkpoints?") rather than open-ended "what do you think?"

Alignment with Course Positioning

These objectives serve the positioning statement: CRE professionals who have used chatbots but aren't getting reliable, production-quality results from AI. The course bridges the gap between single-turn prompting and multi-step agentic workflows by teaching engineering principles through a management lens — while being honest about where the CRE parallel stops and genuinely new AI-specific skills begin. Every objective connects to CRE-specific tasks and contexts. The capstone produces a working CRE workflow plugin that participants refine independently after the course — the beginning of a practice, not a finished product.

The management discipline transfer is bidirectional: participants leave not only with agentic engineering capability but with a more structured approach to human team management — defining success criteria before delegation, decomposing tasks with review gates, and providing specific diagnostic feedback rather than vague direction. The closing exercise makes this reverse transfer explicit.

Next Steps

1. ~~Define learning objectives~~ — Complete (v0.3.0)
2. ~~Incorporate pedagogical review~~ — Complete (v0.2.0)
3. ~~Design assessments~~ — Complete (`03-assessment/rubrics.md` v0.1.0)
4. Plan learning activities: `/generate-outline` then `/generate-lesson-plans` — NEXT
5. Design post-course reinforcement plan (Week 1 Challenge, spaced repetition sequence)
6. Determine final course duration and format (post-lesson-plan)

Version History

- v0.1.0 (2026-02-06): Initial 11 objectives
- v0.2.0 (2026-02-20): Incorporated pedagogical review. Reduced to 9 objectives. Absorbed Six Management Skills into existing objectives. Distributed Documentation Chain (PRD→Obj 3, ADR/DDD→Obj 6, TDD→Obj 8). Added Level 0 orientation, 3 integration pauses, 2 affective objectives, formative/summative designations, "genuinely new" markers. Reframed capstone as Apply/Create "workflow design draft." Time budget reallocated for consolidation.
- v0.2.1 (2026-02-23): Renamed DDD from "Domain Design Document" to "Domain Data Dictionary." Integrated OFD into Obj 3 PRD (3 fields: primary objective, hard constraints, escalation triggers). Added productive failure exercise to Obj 6. Added TDD/EDD distinction to Obj 8 (building inspection vs. appraisal parallel). Updated all documentation chain tables and mappings. Added EDD row to distribution table.
- v0.2.2 (2026-02-23): P0 fixes from learning-objectives-review.md. Corrected time budget arithmetic (585 min instruction, 650 min total — was overstated by 40 min). Added frontier boundary map to Obj 9 objective statement (was in assessment method but not statement). Aligned prerequisite language across all planning documents. Added expanded format note per Gratton directive. Bumped YAML version to match content.
- v0.2.3 (2026-02-23): Claude Code component reframe + capstone upgrade. Framework 5 reduced from 5 to 4 component types (dropped Tools/Scripts, renamed Commands → Slash Commands). Obj 6 updated: added plugin structure introduction, component hierarchy teaching note, strengthened productive failure connection to capstone. Obj 9 reframed from "Workflow Design Draft" to "CRE Workflow Plugin" — capstone is now a self-contained plugin with plugin.json manifest and at least one skill, not a design document. Documentation chain mapping to plugin development added. Summary table, scaffolding plan, and alignment paragraph updated.
- v0.3.0 (2026-02-25): Added Objective 3 (Value-Delivery Fit: Paired VPD Lite + BMC Awareness, 75 min). Renumbered Obj 3-9 → Obj 4-10. Four Pillars ↔ Engineering Substrate bridge connects Obj 2 diagnostic lens to Obj 3 design lens. VPD added as first link in documentation chain (VPD → OFD/PRD → ADR → DDD → TDD). Updated all cross-references: summary table, time budget (585→660 min instruction, 650→725 min total), cognitive distribution, scaffolding plan, management skills map, documentation chain distribution, assessment philosophy. Design: ../../../2026-02-25-bmcvpd-curriculum-integration-design.md

- v0.4.0 (2026-02-25): Added lease abstraction as running anchor example ("I do → We do → You do"). Every objective gets a Lease Abstraction Thread section showing concept-to-application mapping. Level 0 demo updated to use lease abstraction. Obj 6 expanded with five context overflow strategies (knowledge distillation, multi-pass, dynamic loading, chunking, budgeting). Obj 7 productive failure exercise specified as lease abstraction. Obj 10 capstone framing updated with cumulative thread table. No changes to objective statements, time allocations, or assessment methods.
- v0.5.0 (2026-02-25): Updated format from 2-day to 4-day two-weekend (Sat-Sun + week gap + Sat-Sun). Duration updated to ~17 hours total (~12 hours core instruction + ~5 hours extended practice & bookends). Scaffolding plan rewritten for Weekend 1 Days 1-2 + Week Gap + Weekend 2 Days 3-4. All Module fields updated to Weekend/Day references. Management Skills Integration Map updated. Affective objectives updated to Weekend 1 / end-of-course milestones.
- v0.5.1 (2026-02-25): Cross-file deduplication. Obj 4 intent engineering rationale trimmed (detailed OFD-as-intent mapping, progressive specification sentence → cross-ref to teaching-notes.md §Objective 4 where full teaching sequence lives).
- v0.5.2 (2026-02-25): Within-file deduplication. Industry vocabulary note trimmed to era names with cross-ref to Level 0 definitions. Pedagogical Approach overview trimmed to cross-ref §Documentation Chain Distribution. Management Skills Integration Map opening trimmed (principle already stated in Overview line 47).
- v0.6.0 (2026-02-25): Added reverse transfer exercise (5 min) to closing session in scaffolding plan — participants name one human-team management practice they'll change. Added bidirectional transfer paragraph to Alignment with Course Positioning section.